

Islamic University of Technology

Lab 5: Dynamic Programming Engineering & Live Profiling

CSE 4810: Algorithm Engineering

Coin Change

ASH, ABT(PT)

July 16, 2026

General Instruction

This lab consists of **four independent tasks**. Unlike previous labs, written derivations are replaced by **live profiling, execution, and verbal defense**. For every task you must:

- Step 1.** Complete the `// TODO` sections in the provided C++ source file.
- Step 2.** Compile and run the program, feeding it the exact stress-test inputs specified.
- Step 3.** Produce the requested live artifact — a runtime plot, a memory log, or a comparison count — and keep your terminal/IDE open.
- Step 4.** Be ready to defend your engineering choices verbally using the artifact on screen; no written report is collected.

Files Provided

<code>CoinChangeDP.cpp</code>	Greedy vs. DP minimum-coin skeleton
<code>plotter.py</code>	Plots runtime/memory data (auto-detects by filename)

Task 3: Denomination Analysis in the BOUNDED Coin Change Problem

Objective: Prove the mechanical breakdown of greedy heuristics — now made worse by **limited coin supply** — and evaluate spatial locality at the hardware cache level.

Every coin denomination now has a **finite number of available copies**, `counts[i]`. This is no longer the classic unlimited-supply coin change problem: a greedy choice that would always be "safe" when supply is infinite can now leave you **unable to finish at all**, not merely suboptimal.

Variant	Strategy	Why It Matters
coin_change_greedy_bounded	Repeatedly take as many of the largest denomination as both the remaining amount <i>and</i> its remaining supply allow	Can get stuck. A coin choice that looks safe ignores that it may exhaust a small denomination needed later, leaving leftover amount with no usable supply.
coin_change_dp_bounded	Bounded-knapsack-style DP: each coin's contribution is applied at most <code>counts[i]</code> times	Always finds the true optimum (or correctly reports impossibility), because it never commits early the way greedy does.

0.1 TODO 1: Implement `coin_change_greedy_bounded(amount, coins, counts)`

0.2 TODO 2: Implement `coin_change_dp_bounded(amount, coins, counts)`

Sanity Check — Expected Output

```

1 --- BOUNDED COIN CHANGE SANITY CHECK ---
2 Coins {1x5, 3x1, 4x1}, Amount = 10:
3 Greedy : 5 coins (4 + 3 + 1 + 1 + 1)
4 DP      : 5 coins
5 -----

```

Live Experimentation Setup

Runs to Perform

- **The Greedy Dead-End Test:** Coins $\{20, 15, 5, 4, 1\}$ with supply limits $\{1, 1, 2, 2, 2\}$, target amount 23. Greedy takes the 20 first, leaving 3 remaining — but only two 1-coins are left, so greedy **cannot finish**. The true optimum, $15 + 4 + 4 = 23$ (3 coins), never touches the 20 at all.
- **The Benchmark Test:** Coins $\{1, 5, 10, 25\}$ with generous but still finite supply limits $\{20, 10, 10, 5\}$, target amount 99.

Compile & Run

```

1 g++ -O2 -o coinbounded CoinChangeBounded.cpp
2 ./coinbounded
3 python3 plotter.py data_coinbounded.txt

```

Core Engineering Defense (Verbal Check)

- **Heuristic Breakdown:** Show the output for the Dead-End Test. Explain why bounded supply turns a merely suboptimal greedy choice into an **infeasible** one, and why only the DP's optimal-substructure recurrence is guaranteed to find (or correctly rule out) a solution.

- **Why Not Just Backtrack?** Argue why "add backtracking to the greedy loop" is not actually simpler than the bounded DP — in the worst case it re-explores the same exponential search space DP already collapses via memoized subproblems.
- **Cache Locality Breakdown:** Step line-by-line through the bounded DP's inner loop and explain how contiguous array indexing maps to CPU cache lines. Contrast this verbally with how a pointer-heavy tree or linked-list implementation would suffer from cache misses.

Live Challenge The Greedy Dead End Drill

The instructor may ask you to run `coin_change_greedy_bounded` on the Dead-End Test case and show it returning `-1`, then run `coin_change_dp_bounded` on the *same input* and show it returning `3`. You must explain, out loud, why the correct fix is not "make greedy smarter with one exception rule" but a full change in approach — because bounded supply couples every denomination's choice to every other denomination's remaining stock, which is exactly the kind of overlapping-subproblem interaction DP is built to handle.